

Assignment 2 (10% of total marks)

Due date: Sunday, 10 May 2026 by 9:00 pm Singapore time

Scope:

The tasks of this assignment cover **stored PL/pgSQL procedure, function, and trigger**. The assignment covers the topics discussed in lecture 5, 6, 7, and 8.

Assessment criteria:

Marks will be awarded for:

- Correct,
- Comprehensive, and
- Appropriate

application of the materials covered in this subject.

Assignment Specification:

Preliminary actions

Step 1

If you have not done it yet, download the TPCHR sample database and load the sample TPCHR using user account tpchr. This will make the user account tpchr the owner of the TPCHR database.

Step 2

In this step we shall use the relational tables included in a sample TPCHR benchmark database owned by the user tpchr. The conceptual schema of the sample database is included in the file tpchr.pdf.

Task 1 (7.0 marks)

Stored PL/pgSQL Procedure

Main learning objectives

- COUNT()
- grouping by supplier
- average comparison
- handling integer-based aggregates

Task Specification:

Implement a stored PL/pgSQL procedure **SUPPLIERPARTCOUNT** that lists suppliers in a specified region and counts how many different parts each supplier supplies.

The procedure should calculate the average number of parts supplied per supplier in each nation and indicate whether each supplier supplies above or below the nation average.

Input

```
CALL SUPPLIERPARTCOUNT('ASIA');
```

Output should include

Nation name: VIETNAM

Supplier name	Phone	Number of parts supplied	Comment
Supplier#000000884	31-445-893-2210	34	The supplier supplies above the nation average of 28 parts.
Supplier#000001022	31-288-154-9911	21	The supplier supplies below the nation average of 28 parts.

Tables involved

- supplier
- partsupp
- nation
- region

Submission Instructions:

Submit two files:

1. A2SolutionT1.sql – Contains the full PL/pgSQL procedure implementation.
2. A2SolutionT1.pdf – Contains screenshots of terminal output, including:
 - SQL statements executed
 - Procedure output
 - Ensure there are no errors in the displayed results

SQL Script Requirements:

- At the beginning of your script, include the following:
 - **\set ECHO all** – This will ensure that all SQL commands are echoed to the terminal.
 - **\set ECHO none** – Use this to stop echoing the commands when needed.

Important Notes:

- Your **report must satisfy all requirements**; any submission failing to meet them may receive **no marks**.
- Format monetary values consistently using TO_CHAR or equivalent functions (if any).
- Handle **invalid or non-existent regions** gracefully with an informative message.
- Include **all SQL statements and execution output** in your report.

Task 2 (3.0 marks)

Stored PL/pgSQL Function

Task Specification:

Objective:

Implement a stored PL/pgSQL function that finds the **cheapest** and **most expensive** parts within a specified part type.

The function should accept a part type as its input parameter and return a formatted string containing both results.

Scenario and Requirements

You are to create a stored function GET_PARTPRICE_RANGE that performs the following:

1. **Accepts** a part type (e.g., 'STANDARD POLISHED STEEL') as its input parameter.
2. **Identifies** the part with the *lowest* retail price and the part with the *highest* retail price within that type.
3. **Returns** a formatted string displaying:
 - The part key
 - The part name
 - The retail price for both the cheapest and the dearest parts

Expected Output Example

Cheapest part: 143, Part#000000143, \$57.25.

Most expensive part: 1225, Part#000001225, \$986.75.

Assumptions

- The database schema contains the TPC-H-style table:
- part(p_partkey, p_name, p_type, p_retailprice, ...)
- The p_retailprice field represents the retail price of the part.
- You may assume that at least one part exists for a given type.

Example Usage:

```
-- Example calls to the function
SELECT get_partprice_range('ECONOMY POLISHED STEEL');
SELECT get_partprice_range('STANDARD POLISHED STEEL');
SELECT get_partprice_range('PROMO BRUSHED COPPER');
```

Example Output

Cheapest part: 143, Part#000000143, \$57.25.

Most expensive part: 1225, Part#000001225, \$986.75.

Execution Instructions:

- **PL/pgSQL Script:**
 - Write and save the stored function implementation in a PL/pgSQL script named A2SolutionT2.sql.
- **Execution:**
 - Run the function using a SELECT statement to perform the computations.
- **Execution Output:**
 - Capture all terminal output, including success and error messages, in a file named: A2SolutionT2.txt
 - Save the captured output in a **PDF file**: A2SolutionT2.pdf

Submission Instructions:

You are required to submit two files:

1. **SQL script file** (A2SolutionT2.sql) containing:
 - CREATE FUNCTION statement
 - The test SELECT statements
2. **Execution report** showing:
 - Successful creation of the function
 - Results of all test queries
 - No syntax or runtime errors

Report Guidelines:

- Your report must include:
 - **Screenshots of the SQL output:** Copy and paste the screen output from your terminal into a document, then save it as a PDF file.
 - **SQL statements processed:** The report must list **all SQL statements** that were executed.
 - Ensure there are **no errors** in the output shown in your report.

SQL Script Requirements:

- At the beginning of your script, include the following:
 - **\set ECHO all** – This will ensure that all SQL commands are echoed to the terminal.
 - **\set ECHO none** – Use this to stop echoing the commands when needed.

Important Notes:

- Your **report must satisfy all requirements**; any submission failing to meet them may receive **no marks**.
- Format monetary values consistently using TO_CHAR or equivalent functions (if any).
- Handle **invalid or non-existent regions** gracefully with an informative message.
- Include **all SQL statements and execution output** in your report.

Task 3 (7.0 marks)

Stored PL/pgSQL Trigger

Task Specification:

Scenario:

In the LINEITEM table, l_tax must be between 0 and 1.

You are required to create a **row-level trigger** that:

Validates the l_tax value prior to an insert or update operation. If the value is outside the range [0, 1], it must be normalized: values less than 0 are set to 0, and values greater than 1 are set to 1.

Learning focus:

- BEFORE INSERT OR UPDATE
- Numeric validation
- Trigger-based correction of invalid data
- Modifying NEW values

Trigger Details:

- The trigger should fire **before a new lineitem record is inserted or an existing record is updated**.
- It should inspect the l_tax value:
 - If NEW.l_tax < 0 → set NEW.l_tax := 0;
 - If NEW.l_tax > 1 → set NEW.l_tax := 1;
 - Otherwise, leave the value unchanged.

Example output behavior:

- INSERT INTO lineitem(l_orderkey, l_linenum, l_tax) VALUES (1, 2, -1.2);
- -- After trigger: l_tax = 0
- UPDATE lineitem SET l_tax = 100 WHERE l_orderkey = 3 and l_item = 6;
- -- After trigger: l_tax = 0

Execution Instructions:

1. PL/pgSQL Script:

- Implement the trigger in a script named: A2SolutionT3.sql.

2. Execution:

- Test the trigger with at least **two scenarios**:
 - Insert a new lineitem with a negative l_tax.
 - Update an existing lineitem with a positive l_tax greater than 1.

3. Execution Output:

- Capture all terminal output including:
 - Successful insert/update operations that do not fire the trigger.
 - Insert/update operations that fire the trigger.
- Save the output in: A2SolutionT3.txt
- Convert the captured output into a **PDF file**: A2SolutionT3.pdf

Submission Instructions:

You are required to submit two files:

1. **SQL script file** (A2SolutionT3.sql) containing:

- The CREATE TRIGGER statement
- The test INSERT and UPDATE statements to demonstrate trigger behaviour.

2. **Execution report** (A2SolutionT3.pdf) containing:

- Successful creation of the trigger
- Results of all tests
- No syntax or runtime errors

Report Guidelines:

- Your report must include:
 - **Screenshots of the SQL output:** Copy and paste the screen output from your terminal into a document, then save it as a PDF file.
 - **SQL statements processed:** The report must list **all SQL statements** that were executed.
 - Ensure there are **no errors** in the output shown in your report.

SQL Script Requirements:

- At the beginning of your script, include the following:
 - **\set ECHO all** – This will ensure that all SQL commands are echoed to the terminal.
 - **\set ECHO none** – Use this to stop echoing the commands when needed.

Important Notes:

- The trigger should **only modify I_tax** when a value that is not between 0 and 1 is inserted or updated.
- Include **all SQL statements and execution output** in your report.
- Any submission failing to meet these requirements may receive **no marks**.

Submissions

This assignment is due by 9:00 pm (2100 hours) Sunday, 10 May 2026, Singapore time.

Submit the files **solution1.sql, solution1.pdf, solution2.sql, solution2.pdf, solution3.sql, and solution3.pdf** through Moodle in the following way:

- 1) Zip all the files (solution1.sql, solution1.pdf, solution2.sql, solution2.pdf, solution3.sql, and solution3.pdf) into one zipped folder. Name your zipped file as YourPUStudentNumber-A3)
- 2) Access Moodle at **<http://moodle.uowplatform.edu.au/>**
- 3) To login use a Login link located in the right upper corner the Web page or in the middle of the bottom of the Web page
- 4) When successfully logged in, select a site CSCI235 (SP226) Database Systems
- 5) Scroll down to a section Submissions of Assignments
- 6) Click at Submit your Assignment 1 here link.
- 7) Click at a button Add Submission
- 8) Move the zipped file created in Step 1 above into an area provided in Moodle. You can drag and drop files here to add them. You can also use a link *Add...*
- 9) Click at a button Save changes,
- 10) Click at check box to confirm authorship of a submission,
- 11) When you are satisfied, remember to click at a button Submit assignment.

A policy regarding late submissions is included in the subject outline. Only one submission per student is accepted.

Assignment 2 is an individual assignment, and it is expected that all its tasks will be solved individually without any cooperation with the other students. Plagiarism is treated seriously. Students involved will likely receive zero. If you have any doubts, questions, etc. please consult your lecturer or tutor during lab classes or over e-mail.

End of specification